BIG DATA ANALYTICS

Assignment 2: Real-Time Taxi Stream Analytics

Apache Kafka + Streaming Algorithms + LSH

Recommended processing engines: Apache Flink, Spark Structured Streaming, or Kafka Streams

Core requirement: Kafka is mandatory. After the initial producer, every major processing stage must read from a Kafka topic and write its output back to a Kafka topic. A solution that reads the taxi file directly inside Spark/Flink and never uses Kafka between processing stages will not satisfy the assignment.

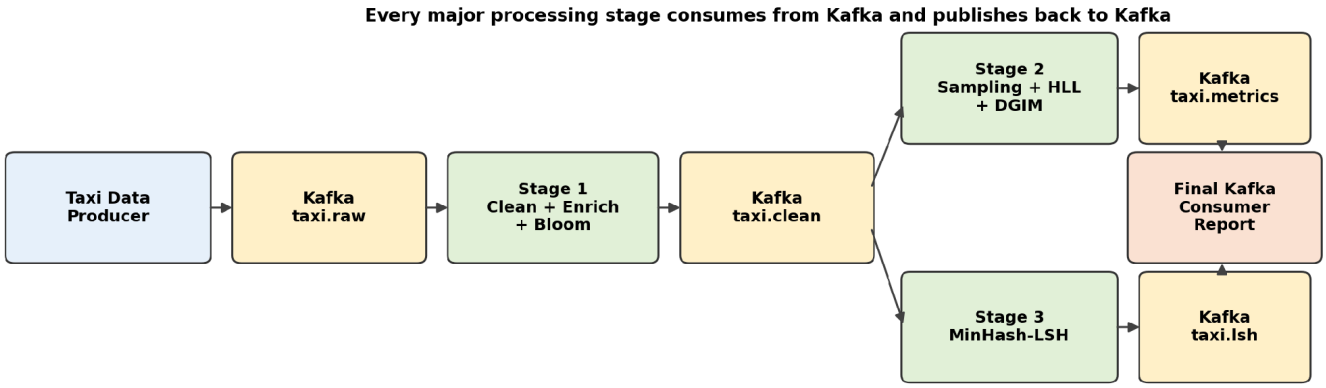


Figure 1. Required logical architecture. Topic names are fixed to simplify evaluation.

Learning goal: Build a reproducible end-to-end streaming system and experimentally study state, approximation, partitioning, producer/consumer speed, latency, throughput, and result quality. LLM use is allowed and expected, but the submitted system must be understood and demonstrable by the team.

1. Assignment at a Glance

You will replay taxi-trip data as a live event stream and build a multi-stage Kafka pipeline. The same stream will be used for all tasks so that the assignment behaves like one application rather than a collection of unrelated programs.

Stage	Consumes	Produces	Required work
Producer	Taxi CSV/Parquet	taxi.raw	Replay trips at a controllable rate; inject duplicates and delayed events.
Stage 1: Clean + Enrich	taxi.raw	taxi.clean	Validate fields, create derived fields, and run Bloom-filter membership test.
Stage 2: Stream Analytics	taxi.clean	taxi.metrics	Sampling, HyperLogLog/FM, DGIM, exact baselines, performance counters.
Stage 3: Similarity	taxi.clean	taxi.lsh	Build destination sets by pickup zone and run MinHash-LSH.
Final Consumer	taxi.metrics + taxi.lsh	Console + JSONL/CSV	Generate the standardized final output used by the TA.

Do not invent alternative topic names or output formats unless there is a technical reason. The fixed structure is intentional. It allows a TA to understand and execute any group submission quickly.

2. Dataset and Event Schema

Use NYC-style yellow/green taxi trip data or the taxi dataset/subset supplied with the course. A static file is acceptable only as the source of the producer: after that point, processing must happen through Kafka.

2.1 Minimum fields used from the taxi data

Field	Meaning / use
pickup_datetime	Event time of the trip.
dropoff_datetime	Used to calculate trip_duration_min.
PULocationID	Pickup zone; also used for partitioning and LSH grouping.
DOLocationID	Drop-off zone; used to define route and destination sets.
trip_distance	Used in exact/sample averages.
fare_amount	Used in exact/sample averages.
passenger_count	Optional descriptive metric.
payment_type	Optional categorical metric.

If your source uses different column names, map them to the names above in the producer. Remove rows with missing pickup/drop-off zone or invalid timestamps.

2.2 Required event published to taxi.raw

```
{
  "event_id": "2026-01-row-00001234",
  "event_time": "2025-01-03T10:21:04",
  "pickup_zone": 161,
  "dropoff_zone": 236,
  "trip_distance": 3.8,
  "fare_amount": 21.50,
  "passenger_count": 1,
  "payment_type": 1,
  "producer_ts_ms": 1760000123456
}
```

- event_id must be stable. If the producer deliberately sends a duplicate, the duplicate must carry the same event_id.
- producer_ts_ms is wall-clock ingestion time and is used for end-to-end latency measurement.
- event_time is the taxi pickup time from the dataset and is used for event-time windows/watermarks.

3. Common Setup and Configuration

Use a single configuration file so that the TA can modify important parameters without editing source code. YAML, JSON, .env, or command-line arguments are acceptable.

```
partitions: 4
producer_rate: 1000          # events/second
duplicate_rate: 0.02         # 2% deliberate duplicate sends
delayed_event_rate: 0.02     # 2% emitted late
max_delay_seconds: 20
window_minutes: 60
hll_precision: 12
dgim_max_window: 10000
bloom_expected_items: 200000
bloom_false_positive_target: 0.01
lsh_num_hashes: 100
lsh_bands: 20
lsh_rows_per_band: 5
jaccard_threshold: 0.50
```

Laptop-scale is sufficient. You do not need a distributed cluster. Docker Compose/local Kafka plus a locally running Spark/Flink/Kafka Streams application is acceptable. Cloud deployment is optional and receives no extra marks by itself.

4. Task 1 - Kafka Producer and Multi-Stage Pipeline

Create the topics below. In the default run, taxi.raw and taxi.clean use 4 partitions. taxi.metrics and taxi.lsh may use 1 partition because they carry compact summaries. You will later rerun selected experiments with different partition counts for taxi.raw/taxi.clean.

```
taxi.raw
taxi.clean
taxi.metrics
taxi.lsh
```

4.1 Producer

1. Read taxi records in timestamp order and publish them to taxi.raw at a configurable rate.
2. Use pickup_zone as the Kafka message key in the baseline implementation.
3. Duplicate approximately 2% of emitted events by resending the same event_id.
4. Delay approximately 2% of events by 1-20 seconds so that event-time handling can be observed.
5. Record actual produced events/second, not only the requested rate.

4.2 Stage 1 - Clean, enrich and publish again

A consumer/stream-processing job must consume taxi.raw and publish one cleaned/enriched event to taxi.clean for each accepted input event. Add these fields:

```
route_key             = "<pickup_zone>-<dropoff_zone>"
trip_duration_min      = (dropoff_datetime - pickup_datetime) in minutes
slow_trip_bit          = 1 if trip_duration_min > 30 else 0
bloom_seen_before      = result of Bloom-filter membership test
stage1_ts_ms           = wall-clock timestamp when Stage 1 emits the record
```

Important Bloom-filter rule: Do not delete an event only because the Bloom filter says "seen before". A Bloom filter can produce false positives. Mark the event and evaluate the decision against ground truth; dropping is allowed only as a clearly identified optional experiment.

5. Task 2 - Streaming Algorithms on taxi.clean

Stage 2 consumes only from taxi.clean. It must periodically publish a compact JSON summary to taxi.metrics. All exact calculations below are evaluation baselines; the streaming estimate is the algorithm of interest.

5.1 Bloom Filter - duplicate membership

The Bloom filter is maintained in Stage 1 using event_id. For every event, record whether the filter says that the event was previously seen.

- Report number of inserted IDs, bit-array size, number of hash functions, observed false-positive rate, and observed false-negative rate.
- Ground truth is available because the producer knows which event_ids it intentionally resent.
- Include at least one concrete false-positive example if one occurs. If none occurs in the default run, increase the number of items or reduce the Bloom-filter size in a separate small experiment until a false positive can be demonstrated.

5.2 Reservoir sampling - a linear statistic

Maintain a reservoir sample of taxi.clean events and estimate the mean fare_amount. Compare it with the exact streaming mean.

- Use reservoir sizes k = 100, 1,000, and 10,000 (or the largest feasible values up to the stream size).
- For each k report exact mean, estimated mean, and percentage error.
- Repeat the same comparison for mean trip_distance.

5.3 Event sampling versus entity sampling - a non-linear statistic

For every 60-minute event-time window, define a pickup zone as diverse if it has trips to at least 10 distinct drop-off zones. Estimate:

```
fraction of active pickup zones in the window that are diverse
```

Compare the following two approaches against the exact answer:

- Event sampling: sample individual taxi events and calculate the property from only the sampled events.
- Entity sampling: select pickup_zone values using a deterministic hash sample and retain all events for the selected zones within the window.
- Use approximately the same storage budget for the two approaches and explain which estimate is more reliable and why.

Purpose of this task: The goal is not merely to produce two numbers. It is to connect sampling design with whether the target statistic is linear/additive or depends on complete behavior of an entity.

5.4 HyperLogLog or Flajolet-Martin - distinct routes

Within each 60-minute event-time window, estimate the number of distinct route_key values, where route_key = pickup_zone-dropoff_zone.

- Maintain an exact set only for ground-truth evaluation.

- Use HyperLogLog if your framework/library provides it. A correct Flajolet-Martin implementation is also acceptable.
- Report exact count, estimate, percentage error, sketch parameters, and approximate memory required by exact vs sketch approach.

5.5 DGIM - recent slow trips

Process slow_trip_bit as the binary stream. Maintain DGIM for the most recent 10,000 cleaned events.

- At report time estimate the number of 1s in the most recent k = 1,000, 5,000, and 10,000 events.
- Compare each estimate with the exact answer and report absolute and percentage error.
- Report the number of DGIM buckets currently stored.
- DGIM must be implemented by the group; using an LLM to help write/debug it is permitted.

6. Task 3 - LSH on the Same Taxi Stream

This task connects the next course topic, Locality-Sensitive Hashing, to the streaming pipeline. Stage 3 consumes taxi.clean and publishes results to taxi.lsh.

6.1 Similarity definition

At the end of every 60-minute event-time window, construct one destination set for each sufficiently active pickup zone P:

```
S(P) = { dropoff_zone values observed for pickup zone P during the window }
```

Only pickup zones with at least 20 trips in the window need to be considered. For two pickup zones P and Q, similarity is Jaccard similarity between S(P) and S(Q). A pair is considered truly similar when Jaccard similarity >= 0.50.

6.2 Required MinHash-LSH procedure

1. Create a MinHash signature for each destination set. Default: 100 hash values.
2. Apply banding. Default: 20 bands x 5 rows = 100 signature values.
3. Generate candidate pickup-zone pairs from shared buckets.
4. For candidate pairs, calculate exact Jaccard similarity and retain pairs whose exact Jaccard similarity is at least 0.50.
5. For evaluation only, calculate all-pairs exact Jaccard on the active pickup zones for that window and identify the true similar pairs.

6.3 LSH metrics

Metric	Definition
Recall of LSH	true similar pairs returned by LSH / all true similar pairs
Candidate precision	true similar pairs among LSH candidates / number of LSH candidates
Candidate reduction	1 - (LSH candidate pairs / all possible pairs)
LSH latency	time to build/query signatures and candidates at window close
Exact latency	time for all-pairs exact Jaccard baseline

- Run at least two LSH parameter settings. One must be the default 20 x 5 setting; choose one different valid band/row configuration that still uses 100 hash values.
- Explain the observed trade-off between candidate count, recall, and processing time.

LSH is not used to approximate Jaccard itself in this task. LSH is used to avoid comparing every possible pair. Exact Jaccard is calculated only for the candidate pairs, while an all-pairs exact computation is used offline as the ground truth for evaluation.

7. Task 4 - Kafka Partitioning, Producer/Consumer Speed and System Performance

The purpose of this task is to observe how a real streaming pipeline behaves when the input rate, partition count, consumer parallelism, and partitioning key change. Do not claim that more partitions are automatically better: measure what happens.

7.1 Metrics you must instrument

- Producer throughput: events sent per second.
- Stage throughput: events processed per second by Stage 1 and Stage 2.
- Kafka consumer lag for taxi.raw and taxi.clean.
- End-to-end latency: final/report timestamp minus producer_ts_ms. Report p50 and p95.
- Partition skew: fraction of messages in the busiest partition.
- Result quality: HLL error, DGIM error, sampling error, and LSH recall for the run.

7.2 Required partition/consumer experiments

Use the same taxi segment and approximately the same producer rate in all four runs. Start each run with cleared/recreated taxi.raw and taxi.clean topics (or another clean equivalent) so that offsets, lag, and counts from a previous run do not contaminate the next run:

Run	Partitions	Parallel consumers / processing slots	Kafka key
P1	1	1	pickup_zone
P2	4	1	pickup_zone
P3	4	4	pickup_zone
P4	4	6	pickup_zone

- Explain why adding consumers beyond the number of partitions may not increase Kafka consumer-group parallelism.
- Compare P2 and P3 to discuss whether multiple partitions improved throughput when the application actually used parallel consumers.

7.3 Required partition-key experiment

With 4 partitions and the same producer rate, compare:

- K1: key = pickup_zone
- K2: no explicit key / round-robin distribution
- Report partition skew, throughput, and whether your processing framework had to repartition/shuffle for per-pickup-zone state.
- Discuss the trade-off between preserving same-key ordering/locality and creating hot/skewed partitions. The correct conclusion depends on your measured workload and framework.

7.4 Required producer-rate experiment

Keep 4 partitions and 4 consumers/processing slots. Run the pipeline at three input rates:

- Low: a rate at which the entire pipeline comfortably keeps up.
- Medium: a rate close to the sustainable processing capacity of your setup.
- High: a rate at which producer speed temporarily exceeds consumer processing capacity and lag becomes visible.
- Record the actual numerical rates you used. You are not graded on achieving a specific events/second value because machine configurations differ.

Rate	Produced/s	Processed/s	p50 latency	p95 latency	max lag	HLL err.	DGIM err.	LSH recall
Low								
Medium								
High								

Interpret result quality carefully: A slower consumer does not automatically make an approximation algorithm less accurate. It may primarily increase lag and latency. Quality can change if your implementation drops late events, samples under a changing load, loses state, or uses time-window policies affected by delay. Report what you observe and explain the mechanism.

8. Task 5 - Event Time, Delayed Events and Failure

Demonstrate two real streaming issues. Both are mandatory but intentionally small.

8.1 Delayed/out-of-order events

- Use the producer configuration that delays approximately 2% of events by up to 20 seconds.
- Use event-time windows in Stage 2/3. If your framework supports watermarks, configure and report the watermark/allowed-lateness policy.
- Show one example of a late event and state whether it was included, updated a previous result, or was discarded.

8.2 Restart/recovery

- While the producer is running, stop Stage 2 for a short period and restart it.
- Report what happens to Kafka lag, offsets, and application state.
- State whether duplicate processing can occur in your chosen framework/configuration and how you observed or handled it.

9. Standardized Output

The final Kafka consumer must read taxi.metrics and taxi.lsh and append one JSON object per report/window to results.jsonl while also printing a concise version to the console. The following fields are mandatory; additional fields are allowed.

```
{
  "run_id": "P3-medium",
  "window_start": "...",
  "window_end": "...",
  "events_processed": 100000,
  "producer_eps": 1000.0,
  "stage2_eps": 970.0,
  "consumer_lag": 130,
  "latency_p50_ms": 210,
  "latency_p95_ms": 860,

  "bloom": {"fp_rate": 0.009, "fn_rate": 0.0},
  "sampling": {"exact_mean_fare": 18.4, "sample_mean_fare": 18.1, "error_pct": 1.63},
  "hll": {"exact_routes": 9840, "estimate": 9735, "error_pct": 1.07},
  "dgim": {"k": 5000, "exact_ones": 812, "estimate": 790, "error_pct": 2.71},
  "lsh": {"candidates": 142, "true_pairs": 38, "recall": 0.947, "candidate_reduction": 0.991}
}
```

10. Submission

Submit one ZIP or Git repository with the structure below. A long project report is not required.

```
README.md
architecture.png
config.yaml                 # or equivalent
producer/
stage1_clean_bloom/
stage2_stream_analytics/
stage3_lsh/
final_consumer/
results/
    results.jsonl
    experiment_summary.csv
LLM_USAGE.md
run_all.sh                  # or run_all.bat / make / docker-compose equivalent
```

10.1 README.md - maximum 4 pages equivalent

- Exact commands to start Kafka, create topics, start each stage, and reproduce the default run.
- Technology/framework selected and why it was selected.
- Machine configuration: CPU, RAM, operating system, and whether Docker/WSL/cloud was used.
- A one-paragraph description of each required experiment and your main observation.
- Any known limitation or task that is incomplete.

10.2 LLM_USAGE.md

LLM use is expected. Include:

- Names of LLMs/coding assistants used.
- Five important prompts that materially helped the implementation.
- At least one case where the generated code/advice was incorrect, inefficient, or unsuitable for your pipeline, and what you changed.
- One short paragraph describing which parts of the final system you can explain without consulting the LLM.

11. TA Evaluation / Demonstration

Evaluation should be reproducible and short. The TA may perform the following checks in any order:

1. Start the supplied run_all command and verify that records move through taxi.raw -> taxi.clean -> output topics.
2. Change producer_rate and observe throughput/lag.
3. Change the partition count or consumer parallelism for a prepared experiment.
4. Ask for one Bloom/HLL/DGIM exact-versus-approximate result.
5. Change DGIM k or reservoir size and rerun/report the result.
6. Ask the group to display candidate pairs and exact Jaccard for one LSH window.
7. Ask one member to explain why 6 consumers do not necessarily outperform 4 consumers when the topic has 4 partitions.
8. Ask for a small code/configuration change. This is intended to verify understanding of LLM-assisted code, not memorization.

TA-friendly design: The TA should primarily inspect the standardized output and run a few parameter changes. Source-code inspection should be used only when results are inconsistent or understanding is unclear.

12. Evaluation Scheme - 40 Marks

Component	Marks	What is checked
Kafka multi-stage pipeline	6	Correct topic flow; no direct file bypass after producer; reproducible execution.
Streaming algorithms	12	Bloom (2), reservoir + sampling design (3), HLL/FM (3), DGIM (4).

LSH task	5	Correct destination sets, MinHash/LSH candidates, exact Jaccard verification, recall/reduction analysis.
Partitioning + rate experiments	7	Partition/consumer tests, key strategy, throughput, lag, p50/p95 latency, interpretation.
Event time + restart behavior	3	Late-event handling/watermark and restart/offset/state observation.
Results + reproducibility	3	Standard JSONL/CSV output, README, config, experiment table.
Demo + understanding	4	Can explain and modify LLM-assisted solution; understands key results and trade-offs.
Total	40	

13. What Is Mandatory and What Is Optional

Mandatory	Optional / no extra marks by itself
Kafka for inter-stage communication	Cloud deployment
One of Flink / Spark Structured Streaming / Kafka Streams (or approved equivalent)	Using more than one stream-processing engine
Bloom + reservoir/sampling + HLL/FM + DGIM	Building every algorithm completely from scratch
MinHash-LSH taxi similarity task	A different LSH application
Required partition/rate experiments	Very high throughput hardware
Event-time late-event test and one restart	Large fault-tolerant production cluster
Exact baselines for evaluation	Persisting exact state in a production deployment

Allowed library use: Framework/library implementations of HyperLogLog and MinHash are allowed. Bloom-filter libraries are also allowed if their parameters can be reported. DGIM should be implemented by the group. The marks are for correct use, measurements, and understanding rather than the number of lines of code.

14. Final Checklist Before Submission

- ☐ Producer reads taxi data and writes only to taxi.raw.
- ☐ Stage 1 reads taxi.raw and writes taxi.clean.
- ☐ Stage 2 reads taxi.clean and writes taxi.metrics.
- ☐ Stage 3 reads taxi.clean and writes taxi.lsh.
- ☐ Final consumer reads Kafka output topics and creates results.jsonl.
- ☐ Bloom false-positive rate is measured.
- ☐ Reservoir sampling and non-linear sampling comparison are complete.
- ☐ HLL/FM exact-vs-estimate result is present.
- ☐ DGIM answers k = 1,000 / 5,000 / 10,000 queries.
- ☐ LSH recall and candidate reduction are reported.
- ☐ P1-P4 partition/consumer experiments are present.
- ☐ pickup_zone vs round-robin partition-key comparison is present.
- ☐ Low/medium/high producer-rate experiment is present.
- ☐ p50/p95 latency and Kafka lag are reported.
- ☐ Late event and restart behavior have been demonstrated.

- ☐ README contains one-command/default-run instructions.
- ☐ LLM_USAGE.md is included.

15. Notes on Interpretation

The experiments are intended to make you reason about the system rather than chase a particular numerical answer. More partitions can improve parallelism, but skew may offset the gain; a higher producer rate may increase lag and latency without changing final correctness; and sketch error need not depend on event rate. Base conclusions on your measurements. The central question is how correctness, approximation quality, state, latency, and throughput change when the same computation is embedded in a real Kafka-based pipeline.